



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

Drona ESP32

Proiectare cu Microprocesoare

Autor: Păvălău Tudor-Stefan

Grupa: 30236

FACULTATEA DE AUTOMATICA
SI CALCULATOARE

Cuprins

1	Introducere: Scop și Motivație	2
2	Studiu Bibliografic și Analiză Comparativă	2
3	Soluția Propusă și Implementare	2
3.1	Arhitectura Generală	2
3.2	Descrierea Algoritmului de Control	3
3.3	Hardware și Conexiuni	3
3.4	Implementare Software	3
4	Testare și Validare	4
4.1	Problema 1: Comunicarea I2C și Adresarea	4
4.2	Problema 2: Instabilitatea Mecanică ("Flip of Death")	4
4.3	Date Experimentale de Calibrare	4
4.4	Testul "În Mână"	4
5	Raport de Daune și Riscuri Asumate	4
6	Concluzii și Direcții Viitoare	5
6.1	Dezvoltare Ulterioară	5

1 Introducere: Scop și Motivație

Proiectul vizează proiectarea și implementarea unui sistem de control al zborului (Flight Controller) pentru un quadcopter, utilizând microcontroller-ul ESP32. Motivația principală a acestui demers nu este replicarea unei soluții comerciale ”gata făcute”, ci înțelegerea profundă a algoritmilor de stabilizare, a filtrării semnalelor digitale și a protocolului de comunicare cu sistemele de propulsie (ESC).

Domeniul vehiculelor aeriene fără pilot (UAV) este unul vast, însă majoritatea soluțiilor hobby (ex. Betaflight) ascund complexitatea matematică în spatele unor interfețe grafice. Scopul acestui proiect este construirea unui sistem ”de la zero”, care să permită controlul fin al turației motoarelor în funcție de datele furnizate de giroscop, demonstrând capacitatea de a menține orizontalitatea vehiculului într-un sistem instabil mecanic.

Am ales arhitectura ESP32 datorită puterii de procesare (dual-core), care permite rularea buclei de control PID simultan cu un server Web pentru telemetrie, oferind un avantaj net față de plăcile clasice Arduino Uno.

2 Studiu Bibliografic și Analiză Comparativă

În faza preliminară, au fost analizate trei abordări distincte pentru realizarea unității de control. Tabelul de mai jos sintetizează diferențele dintre soluțiile existente și soluția propusă.

Tabela 1: Analiza comparativă a soluțiilor de Flight Controller

Criteriu	Soluție Comercială (ex. Betaflight/F4)	Arduino Uno (At-mega328p)	Soluția Propusă (ESP32)
Cost	Ridicat	Scăzut	Mediu-Scăzut
Flexibilitate	Scăzută (Blackbox)	Ridicată	Maximă
Putere Calcul	Foarte Mare (STM32)	Mică (16 MHz)	Mare (240 MHz)
Conectivitate	Necesită modul extern	Necesită modul extern	WiFi / Bluetooth integrat
Protocol ESC	DSHOT / Multishot	PWM Standard	PWM(writeMicroseconds)
Debugging	Port USB/OSD	Serial Monitor	WiFi Telemetry(Web)

Soluția bazată pe ESP32 a fost aleasă deoarece permite ajustarea parametrilor de zbor ”în aer” (Over-the-Air) prin intermediul unei interfețe Web, eliminând necesitatea conectării fizice prin cablu USB pentru fiecare modificare a constantelor PID.

3 Soluția Propusă și Implementare

3.1 Arhitectura Generală

Sistemul este construit în jurul unui microcontroller ESP32 WROOM, interfațat cu un senzor inerțial (IMU) MPU6050 și patru variatoare de turație (ESC). Configurația aleasă pentru cadru este de tip ”+”, unde motoarele sunt dispuse pe axele cardinale (Față, Spate, Stânga, Dreapta), simplificând ecuațiile de mixare a motoarelor.

3.2 Descrierea Algoritmului de Control

Nucleul software este o buclă de control care rulează la o frecvență ridicată (aprox. 50Hz în modul WebServer, optimizabil). Procesul decizional urmează pași:

1. ****Achiziție Date:**** Citirea valorilor brute de accelerație și giroscop via I2C. 2. ****Filtrare:**** Aplicarea unui *Complementary Filter* pentru eliminarea zgomotului mecanic indus de vibrații.

$$Angle_{filt} = 0.8 \times Angle_{vechi} + 0.2 \times Angle_{nou} \quad (1)$$

3. ****Control P (Proportional):**** Calcularea erorii față de orizontală (SetPoint = 0).

$$Output = K_p \times Error \quad (2)$$

Unde K_p (Gain) a fost setat experimental la valoarea 2.0 pentru stabilitate. 4. ****Mixare:**** Combinarea semnalului de bază (Throttle) cu corecțiile calculate pentru Pitch (Față/Spate) și Roll (Stânga/Dreapta).

3.3 Hardware și Conexiuni

O particularitate a ESP32 este matricea GPIO flexibilă. Spre deosebire de Arduino, pinii I2C nu sunt ficși.

- ****Senzor MPU6050:**** Conectat pe pinii SDA (GPIO 33) și SCL (GPIO 32). Această configurare atipică a necesitat inițializarea explicită a librăriei *Wire*.
- ****Motoare:**** Conectate pe pinii digitali 26 (Față), 27 (Dreapta), 14 (Spate), 25 (Stânga).

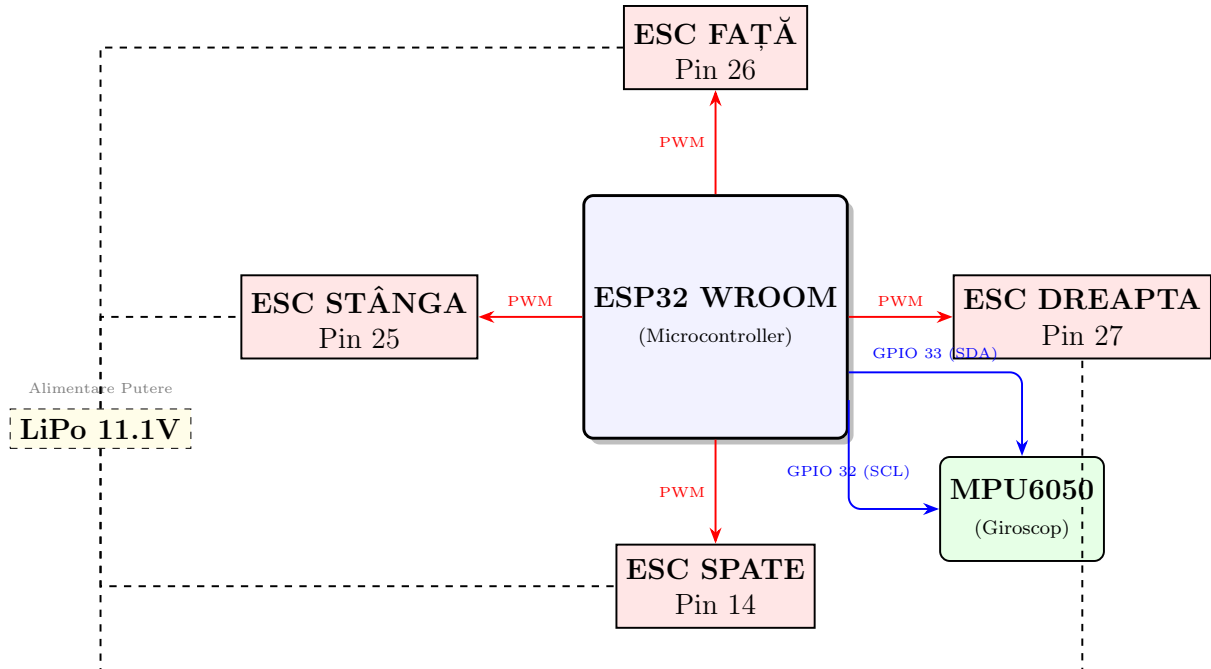


Figura 1: Schema conexiunilor hardware: ESP32, MPU6050 și ESC-uri.

3.4 Implementare Software

Un aspect critic a fost controlul precis al ESC-urilor. Funcția standard `write(0-180)` s-a dovedit inadecvată din cauza rezoluției scăzute. Soluția implementată utilizează funcția `writeMicroseconds(1000-2000)`, care oferă 1000 de trepte de rezoluție și respectă standardul industrial al protocoalelor PWM pentru drone.

4 Testare și Validare

Procesul de validare a fost iterativ, întâmpinând probleme majore specifice sistemelor de zbor.

4.1 Problema 1: Comunicarea I2C și Adresarea

Inițial, senzorul MPU6050 nu a putut fi detectat. Cauza a fost dublă:

1. Bibliotecia standard căuta senzorul pe pinii default (21, 22), în timp ce hardware-ul utiliza (33, 32).
2. Rezolvarea a constat în apelarea explicită: `Wire.begin(33, 32)`.

4.2 Problema 2: Instabilitatea Mecanică ("Flip of Death")

În primele teste de zbor, drona prezenta un comportament violent de răsturnare instantanee la decolare. **Cauza:** Valoarea K_p (Proportional Gain) era setată la 6.0, o valoare prea mare care amplifică vibrațiile mecanice, creând o buclă de feedback pozitiv. De asemenea, lipsa filtrării software permitea zgomotului senzorului să ajungă direct la motoare.

Soluția:

- Reducerea K_p la 2.0.
- Introducerea filtrului complementar (80% istoric / 20% nou) pentru netezirea citirilor.

4.3 Date Experimentale de Calibrare

Tabelul următor prezintă evoluția comportamentului sistemului în funcție de parametrii modificați.

Tabela 2: Evoluția stabilității sistemului în timpul testelor

Fază Testare	P_GAIN	Filtrare	Rezultat Observat
Test Inițial	6.0	Nu	Oscilații violente, Răsturnare (Flip)
Test Intermediar	3.0	Nu	Oscilații rapide, supraîncălzire ESC
Test Corecție Sens	3.0	Nu	Identificare motoare inversate
Configurație Finală	2.0	Da (0.8/0.2)	Decolare stabilă controlată

4.4 Testul "În Mână"

Pentru a valida direcția de corecție fără a risca distrugerea elicelor, s-a efectuat un test static. La înclinarea manuală a dronei spre stânga, sistemul a răspuns prin creșterea turației (RPM) motorului stâng, confirmând logica corectă de stabilizare (motorul "jos" trebuie să accelereze pentru a ridica brațul).

5 Raport de Daune și Riscuri Asumate

Dezvoltarea experimentală a implicat și costuri materiale neprevăzute, cauzate de natura iterativă a procesului de învățare (trial and error) și de erori umane în manipularea componentelor sub tensiune. Pe parcursul proiectului au fost înregistrate următoarele pierderi hardware:

- **2 x ESP32 DEVKITv1:** Arse ireversibil în urma unor conexiuni eronate la pinii de alimentare sau scurtcircuitate pe linia de 3.3V.

- **1 x ESC (Electronic Speed Controller):** Ars ireversibil în urma unei conexiuni eronate.
- **1 x Giroscop MPU6050:** Ars ireversibil.
- **1 x Placă de bază Laptop:** Incident major cauzat de lipsa izolării galvanice în timpul conectării simultane a dronei la bateria LiPo (14.8V) și la portul USB al laptopului pentru debugging.

Acest ultim incident subliniază importanța utilizării unui izolator USB în timpul dezvoltării sistemelor alimentate de surse externe de putere mare.

6 Concluzii și Direcții Viitoare

Proiectul a demonstrat fezabilitatea construirii unui Flight Controller utilizând componente generice și accesibile. Obiectivele inițiale au fost atinse:

- Sistemul citește corect orientarea spațială.
- Comunicarea cu ESC-urile este precisă prin protocolul *Microseconds*.
- Algoritmul de stabilizare reacționează corect la perturbații, evitând fenomenele de rezonanță distructivă prin filtrare software adecvată.

O inovație personală adusă proiectului este interfața Web integrată, care permite vizualizarea în timp real a parametrilor RPM și a înclinării, o funcționalitate care pe sistemele comerciale necesită echipamente de telemetrie dedicate și costisitoare.

6.1 Dezvoltare Ulterioară

Pentru evoluția proiectului de la stadiul de prototip experimental la un vehicul aerian autonom performant, se propun următoarele etape de dezvoltare hardware și software:

1. **Implementarea Controlului prin Joystick Fizic:** Deși interfața Web este utilă pentru telemetrie, pilotajul necesită o latență minimă și precizie tactilă. Se propune dezvoltarea unui transmițător dedicat (bazat tot pe ESP32) care să comunice prin protocolul *ESP-NOW* sau integrarea unui receptor radio clasic (SBUS/PPM).
2. **Menținerea Automată a Altitudinii (Altitude Hold):** Integrarea unui senzor barometric de precizie (ex. BMP280). Prin măsurarea presiunii atmosferice, sistemul va putea ajusta automat accelerația (Throttle) pentru a menține drona la o cotă fixă.
3. **Navigație și Autopilot:** Transformarea dronei într-un sistem autonom prin adăugarea unui modul GPS. Aceasta va permite funcții avansate precum *Return to Home (RTH)* și executarea misiunilor pe puncte (Waypoints).